

PIXOUS TECHNOLOGIES — ESTIMATION & INVOICING API

Security Test Documentation

Authentication, Authorization, Injection & Findings

Generated: August 14, 2026

Source: SECURITY_TESTING.md

Test framework: Pytest + FastAPI TestClient

Contents

1. Framework & conventions

2. How to run

3. Findings fixed in this pass

3.1 JWT accepted usernames that don't correspond to a real account — FIXED · was CRITICAL

3.2 Missing RBAC on PATCH /api/estimations/{id} and PATCH /api/invoices/{id} — FIXED · was HIGH

3.3 QA X-API-Key bypassed authorization entirely — FIXED · was MEDIUM

4. Confirmed-fixed items from the original audit (regression-locked, unchanged this pass)

4.1 test_security_auth.py

4.2 test_security_files.py

4.3 test_security_html_sanitize.py

4.4 test_security_input_validation.py

5. Notable design decisions in the security suite

6. Known gaps / not covered by this suite

Scope: `tests/test_security_auth.py` , `tests/test_security_files.py` ,
`tests/test_security_html_sanitization.py` , `tests/test_security_estimation_invoice_authz.py` ,
`tests/test_security_input_validation.py` , `tests/test_security_qa_api_key.py` .

This document covers **security-focused tests** — authentication, authorization, injection, and file/network-boundary tests — plus the findings this pass fixed directly and why. For pure unit tests of individual modules, see `UNIT_TESTING.pdf` . For end-to-end/feature coverage, see `TESTING.pdf` .

1. Framework & conventions

- **Framework:** Pytest, with FastAPI's `TestClient` (Starlette/httpx) driving the real ASGI app.
- **Isolation:** `tests/conftest.py` points `DATABASE_URL` at a throwaway temp SQLite file *before* any project module is imported; the `fresh_database` fixture drops/recreates every table before each test. No test touches `pixous.db` / production data.
- **Mocking policy:** only external network calls (`requests.get` in `download_url`) are mocked. Authentication, JWT signing/verification, password hashing, role checks, and the database all run for real. A finding is never "proven" against a mock of the thing being tested.
- **Test-naming convention for this suite specifically:** a test whose name describes an attack succeeding (e.g. `..._can_rewrite...` , `..._is_accepted_by...`) documents a live, unfixed gap at the time it was written. Once a gap is fixed, the test is rewritten to describe the *rejection* (e.g. `..._cannot_rewrite...` , `..._is_rejected_by...`) and becomes a regression lock rather than a documented gap. Every finding below states which state each of its tests is currently in.
- **Sensitive-value handling:** `test_catchall_route_leaks_live_dotenv_secrets` deliberately never prints or logs the response body, even on assertion failure — only the *presence* of a leaked secret is asserted, so a regression can't itself leak real credentials into CI logs.

2. How to run

```
# All security tests
python -m pytest tests/test_security_*.py -v

# One file
python -m pytest tests/test_security_estimation_invoice_authz.py -v

# One test
python -m pytest tests/test_security_auth.py::test_login_locks_out_after_repeated_failures -v
```

Current status: **72 passed**, 0 failed.

3. Findings fixed in this pass

Three findings were fixed directly in `api.py` / `config.py` rather than left as documented-but-open gaps, sized to this app's current single-admin (CEO) usage model — the smallest secure change that closes each gap now, without building a multi-tenant RBAC system the product doesn't need yet.

3.1 JWT accepted usernames that don't correspond to a real account — FIXED • was CRITICAL

Root cause. `verify_token` used to check only the HMAC signature and expiry of a bearer token — never that the embedded username corresponded to an actual account. Anyone who knew `JWT_SECRET` (the hardcoded fallback that used to ship in `config.py`, or any leaked value) could mint a valid token for a username that was never created, and pass every protected route with an identity that has no account to disable or audit afterward.

Fix (`api.py`):

- `decode_token(token)` replaces the old boolean-only internals of `verify_token`, returning the decoded `{"user": ..., "exp": ...}` payload (or `None`) so callers can inspect *who* the token claims to be, not just whether the signature checks out.
- `is_valid_username(username, db)` returns `True` only if a `User` row exists for that username, or the username is the bootstrap `config.ADMIN_USERNAME` — the same carve-out `login_endpoint` already trusts before any `User` row exists.
- `auth_middleware` now calls both, for every Bearer-token request, and returns `401` ("Account no longer exists or is inactive.") before the request ever reaches a route handler.
- `get_current_username` performs the same check again independently — defense in depth; it must not rely solely on the middleware having already run.

Known limitation: this checks *existence*, not an *active/disabled* flag — `db.py`'s `User` model has no such column today. If you need to revoke an account without deleting it later, that's a schema addition (`is_active: Boolean`) this pass deliberately didn't make, since existence alone closes the immediate gap.

Tests (all in `test_security_auth.py` unless noted):

Test	Verifies
<code>test_token_forged_for_nonexistent_user_is_rejected_by_middleware</code>	A forged token for a username with no matching <code>User</code> row now gets <code>401</code> , not <code>200</code> (regression lock — previously named <code>..._is_accepted_by_middleware</code> and asserted the opposite)
<code>test_token_for_real_user_is_still_accepted_by_middleware</code>	Positive control: a token for an account that <i>does</i> exist is still accepted
<code>test_token_with_no_matching_user_row_cannot_rewrite_bank_details</code>	The same forged-token scenario against a mutating endpoint (<code>PUT /api/organization</code>) now <code>401</code> s before role is even evaluated
<code>test_token_with_wrong_signature_is_rejected</code>	Tampering the signature of a validly-shaped token still fails (confirms the HMAC check itself, independent of the identity check, was never the weak point)
<code>test_expired_token_is_rejected</code>	A token with <code>exp</code> in the past is rejected
<code>test_protected_routes_reject_missing_auth_header</code>	No <code>Authorization</code> header at all → <code>401</code>
<code>test_decode_token_*</code> , <code>test_is_valid_username_*</code> (<code>test_unit_api_auth_helpers.py</code>)	Direct unit coverage of the two new primitives — see <code>UNIT_TESTING.pdf</code> §5

3.2 Missing RBAC on `PATCH /api/estimations/{id}` and `PATCH /api/invoices/{id}` — FIXED • was HIGH

Root cause. `patch_estimation` and `patch_invoice` resolved the caller's identity (`get_current_username`) but never called `require_role(...)` , unlike every other mutating endpoint in `api.py` (`delete_estimation` , `delete_invoice` , `update_organization` , `upload_organization_asset` , `update_rate_card` , and the dedicated `PUT /invoice/status` endpoint all did). Any authenticated account, regardless of role, could rewrite an estimation's `grand_total` / `timeline_weeks` / `project_name` , or an invoice's `subtotal` / `gst_amount` / `discount` / `total` / `status` . `PATCH /api/invoices/{id}` with `{"status": "Paid"}` achieved the exact same effect as `PUT /api/estimations/{base_name}/invoice/status` , which *did* correctly restrict that action to `Admin` / `Finance` — the two endpoints disagreed on who could mark an invoice Paid.

Fix (`api.py`): both handlers now start with `require_role(request, db, {"Admin"})` , matching the file's existing convention and the current reality that only the CEO's Admin account performs these edits. If Finance is later given day-to-day ownership of invoice edits, widen `patch_invoice` 's allowed-role set to `{"Admin", "Finance"}` to match the status endpoint — left narrower for now rather than guessed at.

Tests (`test_security_estimation_invoice_authz.py`):

Test	Verifies
<code>test_developer_role_cannot_rewrite_estimation_grand_total_via_patch</code>	Developer role → 403 (regression lock)
<code>test_finance_role_cannot_rewrite_estimation_grand_total_via_patch</code>	Finance role → 403 (estimation edits are Admin-only, not Admin/Finance)
<code>test_admin_role_can_still_rewrite_estimation_grand_total_via_patch</code>	Positive control: Admin → 200 , field actually updated
<code>test_token_for_nonexistent_user_cannot_rewrite_estimation_financials</code>	Forged token for a nonexistent user → 401 (caught even earlier, by §3.1's middleware check)
<code>test_patch_estimation_requires_some_valid_auth_token</code>	No auth header → 401
<code>test_patch_estimation_unknown_id_returns_404_not_500</code>	Admin, unknown <code>id</code> → clean 404 , no stack trace in the response
<code>test_developer_role_cannot_mark_invoice_paid_via_patch</code>	Developer role blocked on <i>both</i> <code>PUT /invoice/status</code> (403) and the generic <code>PATCH /api/invoices/{id}</code> (403) — confirms the two endpoints now agree; also confirms the invoice's stored status is genuinely unchanged after both attempts
<code>test_admin_role_can_still_patch_invoice_fields</code>	Positive control: Admin → 200 , status actually updated
<code>test_patch_invoice_requires_some_valid_auth_token</code>	No auth header → 401
<code>test_delete_invoice_still_correctly_requires_admin_role</code>	Positive control: <code>delete_invoice</code> already required Admin before this fix and still does — confirms <code>patch_estimation</code> / <code>patch_invoice</code> are now consistent with it rather than exceptions

3.3 QA `X-API-Key` bypassed authorization entirely — FIXED • was MEDIUM

Root cause. The QA static-API-key branch in `auth_middleware` used to `return await call_next(request)` immediately on a valid key, skipping identity resolution entirely. Any `require_role(...)` check downstream calls `get_current_username`, which requires an `Authorization: Bearer` header — a QA request authenticated only via `X-API-Key` has none. In practice this meant QA traffic either broke unexpectedly against role-gated endpoints, or (had `get_current_username` ever been changed to fail open instead of raising) could have bypassed authorization altogether rather than just authentication.

Fix (`config.py` , `api.py`):

- `config.QA_TEST_USERNAME` added (defaults to `ADMIN_USERNAME`).
- On a valid `X-API-Key` , `auth_middleware` sets `request.state.qa_authenticated_username = config.QA_TEST_USERNAME` and continues — it no longer short-circuits past identity resolution.
- `get_current_username` checks that state first and returns it directly, so `get_current_role` / `require_role` resolve a real role for QA traffic exactly as they would for a Bearer-token login.
- `QA_TEST_API_KEY` remains empty by default — any `X-API-Key` header is rejected with `401` ("QA mode disabled") when it's unset, and `.env.example` documents it as QA/staging-only; leave it blank in production.

Tests (`test_security_qa_api_key.py`):

Test	Verifies
<code>test_qa_api_key_disabled_by_default_rejects_any_header_value</code>	<code>QA_TEST_API_KEY</code> unset → any <code>X-API-Key</code> value still <code>401</code> s
<code>test_qa_api_key_wrong_value_is_rejected</code>	Configured key, wrong value sent → <code>401</code>
<code>test_qa_api_key_correct_value_grants_access</code>	Configured key, correct value sent → <code>200</code>
<code>test_qa_api_key_resolves_to_configured_username_not_a_bypass</code>	<code>QA_TEST_USERNAME</code> pointed at a real Developer-role account → an Admin-only endpoint (<code>PUT /api/rate-card</code>) still <code>403</code> s — the key is not a bypass
<code>test_qa_api_key_with_admin_configured_username_passes_admin_gate</code>	Positive control: <code>QA_TEST_USERNAME</code> pointed at an Admin-role account → the same Admin-only endpoint succeeds
<code>test_qa_api_key_defaults_to_bootstrap_admin_username_with_no_user_row</code>	Default <code>QA_TEST_USERNAME (== ADMIN_USERNAME)</code> works out of the box with no <code>User</code> row provisioned
<code>test_qa_api_key_takes_precedence_over_a_missing_bearer_token</code>	A request with only <code>X-API-Key</code> (no <code>Authorization</code> header at all) is still correctly authenticated, not treated as anonymous

4. Confirmed-fixed items from the original audit (regression-locked, unchanged this pass)

These were already covered before this test pass and remain green — listed here for completeness so this document is the full picture of the app's security test coverage, not just what changed most recently.

4.1 test_security_auth.py

Test	Verifies
test_login_succeeds_with_correct_password_and_stores_bcrypt_hash	Correct login succeeds; the stored <code>password_hash</code> is a real bcrypt hash, never the raw password
test_login_rejects_wrong_password	Wrong password → 401
test_legacy_plaintext_row_still_logs_in_and_is_upgraded_to_bcrypt	A pre-hashing legacy row (raw plaintext in <code>password_hash</code>) still logs in on the correct password, and is transparently rehashed to bcrypt on that first successful login
test_stored_password_is_never_plaintext_after_creation	<code>password_hash</code> for a freshly created user is bcrypt, never plaintext
test_bootstrap_admin_login_uses_whatever_is_actually_configured	The one-time bootstrap admin login works using whatever <code>ADMIN_PASSWORD</code> is actually configured in this environment (skips if unset — the secure default)
test_wrong_password_rejected_on_empty_user_table	Wrong password against the bootstrap path (empty <code>User</code> table) still 401 s
test_bootstrap_path_disabled_when_admin_password_unset	<code>ADMIN_PASSWORD=""</code> fully disables the bootstrap login path
test_login_locks_out_after_repeated_failures	8 rapid failed attempts → first 5 return 401, the rest return 429
test_rate_limit_is_scoped_per_username_not_globally	A lockout on one username doesn't lock out a different legitimate user from the same client IP
test_successful_login_resets_the_failure_counter	A successful login resets the failure count, so the next few wrong attempts don't immediately re-trigger the lockout
test_developer_role_cannot_rewrite_bank_details	Developer role → 403 on <code>PUT /api/organization</code>
test_admin_role_can_rewrite_bank_details	Admin role → 200, bank details actually updated

Test	Verifies
<code>test_developer_role_cannot_delete_estimations</code>	Developer role → 403 on <code>DELETE /api/estimations/{id}</code>
<code>test_finance_role_can_update_invoice_status_but_developer_cannot</code>	Finance role passes the invoice-status role gate; Developer role is blocked
<code>test_cors_never_allows_credentials_regardless_of_origin</code>	<code>allow_credentials</code> is never <code>true</code> in a CORS preflight response, regardless of the <code>Origin</code> sent — auth here is a Bearer header, never a cookie, so credentialed cross-site requests have nothing to ride along
<code>test_deleting_nonexistent_estimation_does_not_leak_stack_trace</code>	A 404/500 error response never contains <code>"Traceback"</code> or a raw <code>sqlalchemy</code> internal string

4.2 `test_security_files.py`

Test	Verifies
<code>test_catchall_path_join_escapes_frontend_dist_for_dotdot_input</code>	Reproduces, in isolation, why an unguarded <code>FRONTEND_DIST / full_path</code> join is unsafe for <code>../../config.py</code> -style input
<code>test_catchall_route_does_not_serve_files_outside_frontend_dist</code>	The actual SPA catch-all route rejects both literal <code>../</code> and percent-encoded (<code>..%2f</code>) traversal attempts, for both <code>config.py</code> and <code>.env</code> targets
<code>test_catchall_route_leaks_live_dotenv_secrets</code>	Regression lock for the specific historical exploit: the encoded-traversal payload must never return the live <code>.env</code> file's contents (asserts <i>absence</i> only — never logs the body, see §1)
<code>test_branding_upload_requires_no_authentication</code> <i>(name reflects the route, not a vulnerability)</i>	<code>POST /api/organization/{slot}</code> with no auth header → 401 (the middleware gate itself; what an authenticated upload accepts is covered separately below)
<code>test_save_branding_file_rejects_html_extension</code>	An upload named <code>signature.html</code> containing a <code><script></code> payload is rejected, never written to disk

Test	Verifies
<code>test_save_branding_file_rejects_content_mismatched_extension</code>	A file named <code>logo.png</code> whose real bytes are HTML (extension/content-type spoofing) is rejected — Pillow verifies the actual format
<code>test_save_branding_file_accepts_a_real_png</code>	Positive control: a genuine PNG upload still succeeds
<code>test_save_branding_file_never_deletes_existing_asset_on_invalid_upload</code>	A rejected upload never deletes the pre-existing valid asset for that slot — validation happens before any file is touched
<code>test_branding_static_mount_requires_no_authentication</code>	<code>/branding/*</code> static files are servable with no <code>Authorization</code> header, by design (needed for the pre-login logo) — documents this is intentional, not a gap, given the upload-side validation above
<code>test_download_url_blocks_internal_and_link_local_targets</code>	Cloud metadata (<code>169.254.169.254</code>), loopback, RFC1918 private ranges, and the <code>file://</code> scheme are all rejected by <code>_assert_safe_url</code> before <code>requests.get</code> is ever called (verified via a spy that fails the test if reached)
<code>test_download_url_allows_public_looking_host</code>	Positive control: a hostname resolving to a public IP is still allowed through
<code>test_download_url_does_not_follow_redirects</code>	A 302 response is never followed (<code>allow_redirects=False</code> is enforced) — closes the classic "public host redirects to an internal address" allow-list bypass

4.3 `test_security_html_sanitize.py`

Test	Verifies
<code>test_strips_script_tags</code>	<code><script></code> content is removed entirely, surrounding content preserved
<code>test_strips_event_handler_attributes</code>	<code>onerror</code> / <code>onclick</code> attributes are stripped; unrelated attributes (<code>src</code>) survive
<code>test_neutralizes_javascript_uri</code>	<code>href="javascript:..."</code> URIs are neutralized
<code>test_strips_iframe_object_embed</code>	<code><iframe></code> / <code><object></code> tags are removed

Test	Verifies
<code>test_preserves_legitimate_invoice_markup_and_styles</code>	Full invoice document structure, <code><style></code> blocks, and inline <code>style</code> attributes survive sanitization unmangled
<code>test_invoice_save_endpoint_strips_injected_script</code>	End-to-end: a <code><script></code> payload submitted through <code>PUT /api/documents/{base_name}/invoice</code> is stripped before being stored in the DB

4.4 `test_security_input_validation.py`

Test	Verifies
<code>test_organization_update_rejects_malformed_email</code>	<code>email</code> failing <code>EMAIL_RE</code> → 422
<code>test_organization_update_rejects_phone_with_wrong_digit_count</code>	<code>phone</code> not exactly 10 digits → 422
<code>test_organization_update_rejects_non_numeric_phone</code>	Non-numeric <code>phone</code> → 422
<code>test_organization_update_rejects_malformed_gstin</code>	<code>gstin</code> failing <code>GSTIN_RE</code> → 422
<code>test_organization_update_accepts_valid_gstin_and_normalizes_case</code>	Valid lowercase GSTIN accepted and upper-cased on save
<code>test_organization_update_html_in_name_field_is_stored_but_never_executed</code>	<code>name</code> field accepts arbitrary text including <code><script></code> — documents that <i>this</i> field's safety comes from render-time escaping (see <code>UNIT_TESTING.pdf</code> §4), not input rejection
<code>test_organization_update_rejects_oversized_address_field</code>	<code>address</code> beyond its <code>max_length</code> → 422
<code>test_organization_update_requires_admin_and_rejects_sql_metacharacters_safely</code>	A classic SQL-injection-shaped string (<code>(Robert'); DROP TABLE ...;--</code>) is stored verbatim via the parameterized ORM call, never interpreted; the table remains queryable afterward
<code>test_patch_estimation_rejects_negative_grand_total</code>	Negative <code>grand_total</code> in a patch payload → 422
<code>test_patch_estimation_rejects_negative_timeline_weeks</code>	Negative <code>timeline_weeks</code> → 422
<code>test_patch_estimation_rejects_missing_required_version_field</code>	Omitting the required optimistic-lock <code>version</code> field → 422

Test	Verifies
<code>test_invoice_patch_rejects_negative_subtotal</code>	Negative <code>subtotal</code> on an invoice patch → <code>422</code>
<code>test_rate_card_update_requires_admin_role</code>	Developer role → <code>403</code> on <code>PUT /api/rate-card</code>
<code>test_rate_card_update_persists_new_custom_role_for_admin</code>	Admin can add a new custom rate-card role; test snapshots/restores the process-global <code>config.DEVELOPER_RATES</code> dict so it can't leak into other tests
<code>test_get_rate_card_is_readable_without_admin_role</code>	Reading the rate card has no role restriction (only writing does)

5. Notable design decisions in the security suite

- **Forged tokens use the real signing path.** Every "attacker" token in this suite is built with the exact same HMAC construction `api.create_token` uses (see `_forge_token` / `auth_helper.forge_token`), so a test failure means the real signing/verification code has a gap — not that the test's forgery routine drifted from production behavior.
- **Regression tests over "documented gap" tests, once fixed.** A test that once proved a vulnerability existed is rewritten in place (not just deleted and replaced) once the fix lands, so `git log` on the test file shows the exploit → fix → lock progression rather than losing that history.
- **Global mutable state is snapshotted and restored.** `update_rate_card` mutates the process-global `config.DEVELOPER_RATES` dict in place; `test_rate_card_update_persists_new_custom_role_for_admin` snapshots it beforehand and restores it in a `finally` block so it can't bleed into other tests' expectations of the default rate card.
- **Sized to actual usage, not maximal RBAC.** The Admin-only fix in §3.2 is deliberately narrower than "Admin or Finance" even though the dedicated status endpoint allows Finance — because no Finance-role workflow for editing estimations/invoices directly exists yet in this product. Widening it later is a one-line change with a clear trigger (a real Finance user needing it), not a guess made now.

6. Known gaps / not covered by this suite

- No `is_active` /disabled flag exists on `User` yet (see §3.1) — a compromised or ex-employee account can only be removed, not deactivated-and-audited-later.
- Session/token revocation: there's no server-side token blocklist — a leaked, unexpired JWT for a real, still-active account remains valid until it naturally expires (24h). This is a separate concern from the "does the account exist" check fixed in §3.1.
- Rate limiting is per-process (`utils/rate_limiter.py` 's own module docstring already documents this); behind multiple workers the effective limit is `MAX_FAILED_ATTEMPTS * worker_count`. Not exercised by any test here since it requires a multi-process setup this suite doesn't run.